Student Services Information System (SSIS)

System Requirements Plan

1. Project Goal

Build one connected web system where students handle **enrollment, clearance, grades, and document requests online**, so they no longer need to line up at school offices. The system has five roles (Student, Registrar, Cashier, Department, and Admin) that all share **one MySQL database**. When staff update something, the student sees it right away.

- **Enrollment queuing** is replaced by online enrollment requests that the Registrar reviews.
- **Clearance** is replaced by an online checklist where each office approves its own part.
- **Grade viewing** is replaced by a student page that shows released grades.
- **Requests** for documents are replaced by an online request, payment check, and "Ready for Release" notice, so students only visit the school when their document is ready.

2. Key Decisions

These choices were open in the first plan. They are now fixed so the team does not argue about them mid-build.

Topic	Decision	Why
Tech stack		
Hosting		
Student accounts	No self-registration. The registrar creates student accounts. The admin creates staff accounts.	Stops fake accounts.
Password reset	No email. A user sends a reset request, and then the admin (or registrar for students) gives a temporary password. The user must change it at the next login.	Free hosts often block email.
Notifications	Inside the system only (bell icon with unread count)	Simple and reliable.
Document release	Pick up at the Registrar's office using the request number.	Matches the "Ready for Release" status.
Payments	The cashier records and verifies payments. A student may type a reference number (for example, from GCash or a bank). No payment gateway and no file uploads.	Keeps scope small and safe.

Topic	Decision	Why
Grades	Draft first, then release. A released grade can be corrected only with a reason, and the change is logged.	Protects grade accuracy.
Approved vs. Enrolled	**Approved** = The registrar accepted the request. **Enrolled** = The registrar confirmed the final subject list.	The first plan did not explain the difference.
Department access	Each department staff account belongs to one department. They see students whose program belongs to that department.	Gives a clear rule for "only relevant students."
Starter data	seed.sql loads departments, programs, subjects, fees, requirements, and test accounts. All screens read from the database.	"No fake data" means no hardcoded screens, not an empty database.
Printing	Clearance, request slip, and receipt use the browser's Print (print-friendly CSS). No PDF library.	Fewer things to install.
Deleting	Records are never deleted. They are set to Inactive or Cancelled.	Keeps history and audit trail intact.

3. Tech Stack and Project Setup

Layer	Choice	Notes
Frontend		
Backend		
Database		
Team tools	XAMPP (local testing), Git/GitHub, phpMyAdmin	Import schema.sql and then seed.sql on every machine so everyone has the same data.

Rules for the whole project

- Use a **responsive design** for desktop, tablet, and mobile.
- Use a **centralized database** with a single connection file. No module may connect on its own.
- Split the project into **separate folders**, one per module (see below).
- Build **reusable components** once and use them everywhere: navigation bar, sidebar, page header, alerts, tables, forms, status badges, and the login/role check.
- Make code **teammate-friendly**: a comment at the top of every file saying what it does, a comment above every function, clear names, and one job per function.

Folder structure

```
ssis/
├── index.php            (sends visitors to login or their dashboard)
├── login.php  logout.php  change_password.php  forgot_password.php
├── config/
│   ├── database.php      (the ONE place that connects to MySQL)
│   └── app.php           (school name, session timeout, lockout limits)
├── includes/             (shared code used by every module)
│   ├── auth.php          (session start, login check, role guard)
│   ├── csrf.php          (form security tokens)
│   ├── helpers.php       (escape output, validate input, badges, dates)
│   ├── audit.php         (writes audit log entries)
│   ├── notify.php        (creates notifications)
│   └── layout/           (header, sidebar, navbar, footer, alerts)
├── modules/
│   ├── student/  registrar/  cashier/  department/  admin/
├── assets/
│   └── css/  js/  img/
└── database/
    ├── schema.sql        (creates every table)
    └── seed.sql          (starter data + test accounts)
```

4. User Roles and Permissions

Each role has its own dashboard, menu, and folder. The matrix below shows who can do what.

Function	Student	Registrar	Cashier	Department	Admin
Log in and change your own password	Yes	Yes	Yes	Yes	Yes
View own dashboard, grades, clearance, and requests.	Yes	—	—	—	—
Search / view student records	—	Yes	Payments only	Own department	Yes
Add/edit student records	—	Yes	—	—	Yes
Review enrollment	—	Yes	—	—	View
Encode and release grades	—	Yes	—	—	—
Review clearance items	—	Own items	Own items	Own items	View
Process document requests	—	Yes	—	—	View
Record / verify payments	—	—	Yes	—	View
Create accounts / reset passwords	—	Students only	—	—	All users

Function	Student	Registrar	Cashier	Department	Admin
Manage departments, programs, subjects, fees, settings	—	—	—	—	Yes
Post announcements	—	Yes	—	—	Yes
Reports	—	Yes	Payments	Pending clearance	All
View audit logs	—	—	—	—	Yes

5. Authentication and Accounts

- **Login** with a username. Students use their student number. Staff use their assigned username.
- **Role-based redirect:** after login, each role goes to its own dashboard.
- **Logout** destroys the session completely.
- **Password hashing:** use password_hash() and password_verify(). Never store plain passwords.
- **Password rules:** at least 8 characters with letters and numbers.
- **First login:** accounts with a temporary password are forced to change it before doing anything else.
- **Change password:** requires the current password.
- **Password reset:** the user submits a reset request, and the admin (or registrar for students) issues a temporary password.
- **Active / inactive accounts:** an inactive user cannot log in. The status is checked on every page load, so a disabled user is cut off right away.
- **Brute-force protection:** after 5 wrong attempts, the account and IP are locked for 15 minutes. Every attempt is saved in login_attempts.
- **Access control:** every page begins with a role check (for example, require_role('registrar')). A user who opens another role's page is blocked, and the attempt is logged.
- **Own records only:** student pages always use the logged-in user's ID from the session. Never trust an ID sent from the browser.

6. Student Portal

Dashboard

Shows, all read from the database: student name, student ID, program, year level, enrollment status, clearance status, recent grades, pending document requests, payment status, and the latest notifications and announcements.

Profile

- View profile. Students can edit only the **contact number, address, and email**. Name, student number, program, and year level are changed by the registrar.
- Change password.

Enrollment

- View enrollment information for the current term.
- Submit an enrollment request: choose year level and pick subjects from the program's subject list. The page shows total units and blocks going over the unit limit set by the admin.
- Only **one active request per term**. A student can cancel while the status is pending.
- View enrolled subjects and the enrollment status. Rejected requests show the reason, and the student may submit a new one.
- **Statuses:** Pending, Under Review, Approved, Rejected, Enrolled, Cancelled.

Grades

- View subject code, subject name, units, grade, grade status (Passed, Failed, Incomplete, Dropped), semester, and academic year. Filter by term.
- Only **released** grades are shown. Draft grades are never visible.
- **A student must never see another student's grades.** Every grade query is limited to the logged-in student.

Clearance

- View the requirement list and each office's status (Pending, Approved, Rejected).
- View the overall status (In Progress, Action Needed, Cleared) and any rejection reasons.
- Resubmit a rejected item after fixing the problem.
- Print the clearance page once it is fully cleared.

Document Requests

- Choose a document type (the fee is shown), enter the purpose and number of copies, and submit. The system creates a **request number**.
- View the request status, payment status, and processing status.

- If a fee applies, type the payment reference number to send it for cashier verification.
- Cancel while the status is Pending or For Payment (not after payment is verified).
- **Document types:** Certificate of Enrollment, Certificate of Grades, Transcript of Records, Good Moral Certificate, and other types the admin adds.
- **Statuses:** Pending, Under Review, For Payment, Paid, Processing, Ready for Release, Completed, Rejected, Cancelled.

Notifications

A bell icon with an unread count on every page. A notifications page lists all messages, lets the student mark them as read, and links each one to the related page. Events are listed in Section 12.

7. Registrar Module

- **Student records:** search, add, edit, and set active/inactive. Adding a student also creates their login account.
- **Enrollment:** queue of requests. Open one, adjust the subject list if needed, then mark Under Review, Approve, Reject (reason required), or Confirm Enrolled.
- **Grades:** pick a term and subject, see the enrolled students, enter grades in one table, save as Draft, then Release (one grade or the whole class). Only students who are **enrolled** in that term can be graded. Editing a released grade needs a reason.
- **Clearance:** review the Registrar's own requirement, approve or reject with a reason, and view the overall clearance of any student.
- **Document requests:** queue with filters. Move requests through the allowed statuses (Section 12), reject with a reason, and mark Completed after pickup.
- **Announcements and notifications:** post announcements and send a message to a student.
- **Reports:** see Section 15.

Search and filter by: student ID, student name, program, year level, request number, and status.

8. Cashier Module

- **Search students** and view their payment records.
- **Payment list** with filters for student, date, status, and request number.
- **Record a payment** (walk-in): choose the request that is for payment, enter the amount (must match the fee), method (cash, GCash, bank transfer, or other), and reference number. The system makes a receipt number.
- **Verify** a payment that a student sent: mark it Paid or Rejected with a reason. A rejected payment goes back to the student to send again.

- **Refund** a paid payment (reason required) when the related request was cancelled or rejected.
- **Transactions page** showing reference numbers, with printable receipt information.
- **Clearance:** review the cashier's clearance item. The page shows whether the student still has unpaid or pending payments.
- **Payment statuses:** Unpaid, Pending Verification, Paid, Rejected, Refunded.
- When a payment becomes paid, the related document request becomes paid automatically, and the registrar is notified.

9. Department Module

- **Dashboard** with the number of pending clearance items for the department.
- **View and search students** from programs under the staff member's department (by student ID, name, program, or year level).
- **View the clearance requirements** that belong to the department.
- **Review clearance:** approve or reject with a required reason.
- **Monitor pending requests** in one list.
- **Limit:** Department staff see only name, program, year level, and clearance details. They cannot see grades, payments, or students from other departments.

10. Admin Module

User management

- Add and edit users, and assign roles and departments.
- Enable or disable accounts. Admins cannot disable their own account.
- Reset passwords (temporary password) and handle reset requests.

System management

- Manage departments, programs, subjects, and terms (including which term is current).
- Manage document types (name, fee, active or not).
- Manage clearance requirements (name, which office reviews it, and active or not).
- Manage system settings: school name, maximum units per enrollment, login attempt limit, lockout time, and session timeout.

Monitoring

- Dashboard with totals: users per role, enrollments, pending requests, and payments.
- Reports (Section 15).
- Audit log viewer filtered by user, action, and date range.

- User activity (last login) and system activity.

11. Database

Design rules

- Primary key on every table.
- Foreign keys for every relationship. Use ON DELETE RESTRICT, since records are never deleted.
- created_at and updated_at timestamps on every main table.
- Money in DECIMAL(10,2). Passwords in VARCHAR(255).
- Status columns use fixed ENUM values with the exact spelling in the table below.
- No duplicate data. For example, the student's enrollment status is read from enrollments, not copied into students.
- Add indexes on student_no, request_no, status, and every foreign key to keep pages fast.

Tables

Table	What it stores	Key links and notes
users	Login accounts for all roles	username (unique), password_hash, role, department_id (staff only), status, must_change_password, last_login
students	Student profile	user_id, student_no (unique), name, program_id, year_level, contact details
departments	Departments	name, code
programs	Programs / courses	department_id, name, code
subjects	Subject list per program	program_id, code, name, units, year_level, semester
terms	Semester and academic year	academic_year, semester, is_current
enrollments	Enrollment requests	student_id, term_id, year_level, status, reviewed_by, reason. One active request per student per term
enrollment_subjects	Subjects in each enrollment	enrollment_id, subject_id
grades	Student grades	student_id, subject_id, term_id, grade, grade status, status (Draft/Released), encoded_by. Unique per student + subject + term

Table	What it stores	Key links and notes
clearance_requirements	What must be cleared	name, office (Department, Cashier, Registrar), department_id, is_active
clearances	One clearance per student per term	student_id, term_id, overall_status
clearance_items	Each office's decision	clearance_id, requirement_id, status, reviewed_by, rejection_reason, reviewed_at
document_types	Available documents	name, fee, is_active
document_requests	Document requests	request_no (unique), student_id, document_type_id, purpose, copies, status, rejection_reason, processed_by
payments	Payment records	request_id, receipt_no, reference_number, amount, method, status, recorded_by, verified_by, verified_at
notifications	In-system messages	user_id, title, message, link, is_read, created_at
announcements	School announcements	title, body, audience, posted_by
audit_logs	Who did what, and when	user_id, action, entity, entity_id, details, ip_address, created_at
login_attempts	Failed and successful logins	username, ip_address, was_successful, attempted_at
password_resets	Reset requests	user_id, status, handled_by, created_at
settings	System settings	setting_key (unique), setting_value

Status values

Item	Allowed values
Enrollment	Pending, Under Review, Approved, Rejected, Enrolled, Cancelled
Grade	Draft, Released. Grade status: Passed, Failed, Incomplete, Dropped
Clearance item	Pending, Approved, Rejected
Overall clearance	In Progress, Action Needed, Cleared
Document request	Pending, Under Review, For Payment, Paid, Processing, Ready for Release, Completed, Rejected, Cancelled
Payment	Unpaid, Pending Verification, Paid, Rejected, Refunded
Account	Active, Inactive

Number formats

- Student number: 2026-00001
- Document request number: DR-2026-0001
- Receipt number: OR-2026-0001

12. Main System Workflows

Each workflow is one connected chain. Every step updates the database, writes an audit log entry, and creates a notification.

1. **Enrollment:** Student submits request → Registrar reviews → Approve or Reject → Registrar confirms Enrolled → Student is notified at each step.
2. **Grades:** Registrar enters the grade (draft) → The registrar releases it → The student can view it and is notified.
3. **Clearance:** Student opens clearance → each office (department, cashier, and registrar) approves or rejects its item → when every item is approved, the overall status becomes **cleared.** If any item is rejected, the status is **Action Needed** until the student resubmits.
4. **Document request:** Student submits → Registrar reviews → Payment (if the fee is more than 0) → Processing → Ready for Release → Completed after pickup. Free documents skip the payment step.
5. **Payment:** Payment recorded or reference number sent → Cashier verifies → payment status updated → the related document request updates automatically.
6. **Notification:** Any important action creates a notification automatically for the right users.

Allowed status changes

Any change not listed here is blocked by the system.

Item	Change	Who	Rule
Enrollment	Pending → Under Review	Registrar	Opening the request
Enrollment	Under Review → Approved	Registrar	Subject list checked
Enrollment	Pending / Under Review → Rejected	Registrar	Reason required
Enrollment	Approved → Enrolled	Registrar	Subjects become official
Enrollment	Pending → Cancelled	Student	Registrar can cancel until Enrolled
Document request	Pending → Under Review	Registrar	—
Document request	Under Review → For Payment	Registrar	Only if fee is more than 0. An Unpaid payment is created

Item	Change	Who	Rule
Document request	Under Review → Processing	Registrar	Only if fee is 0
Document request	For Payment → Paid	System	Happens when the Cashier marks the payment Paid
Document request	Paid → Processing → Ready for Release → Completed	Registrar	Completed after pickup
Document request	Pending / Under Review → Rejected	Registrar	Reason required
Document request	Pending / For Payment → Cancelled	Student	Not allowed once payment is Paid
Payment	Unpaid → Pending Verification	Student or Cashier	Reference number entered
Payment	Pending Verification → Paid / Rejected	Cashier	Reason if Rejected. Student may send again
Payment	Unpaid → Paid	Cashier	Walk-in payment recorded and verified at once
Payment	Paid → Refunded	Cashier	Reason required. Only if request was cancelled or rejected
Clearance item	Pending → Approved / Rejected	Responsible office	Reason required if Rejected
Clearance item	Rejected → Pending	Student	Resubmit after fixing the problem
Overall clearance	In Progress / Action Needed / Cleared	System	Worked out automatically from the items
Grade	Draft → Released	Registrar	Students see Released grades only
Grade	Released → corrected	Registrar	Reason required, student notified, change logged

Notification events

When this happens	Who is notified
Enrollment request submitted	Registrar staff
Enrollment status changes	The student
Grades released or corrected	The student

When this happens	Who is notified
Clearance submitted or resubmitted	The office that must review it
Clearance item approved or rejected; clearance becomes Cleared	The student
Document request submitted	Registrar staff
Document request status changes (including Ready for Release)	The student
Payment reference number sent	Cashier staff
Payment verified, rejected, or refunded	The student (and the Registrar when Paid)
Password reset request sent	Admin
Announcement posted	Everyone in the chosen audience

13. Security

Threat	Protection required
SQL injection	Prepared statements (PDO) for every query. Never put user input directly into SQL.
Brute-force login	5 failed attempts → 15-minute lockout. Login attempts are saved. Error message stays general ("Invalid username or password").
Unauthorized access	Role check at the top of every page and every action, done on the server. Frontend hiding is not enough.
Seeing other students' data	Always filter by the logged-in user's ID from the session. Never trust an ID from the URL or form.
Cross-site scripting (XSS)	Escape all output with htmlspecialchars().
Cross-site request forgery (CSRF)	A hidden token in every form, checked on every POST request.
Session theft	New session ID at login, HttpOnly + Secure + SameSite cookies, 30-minute idle timeout, full destroy on logout.
Stolen database	Passwords hashed. No sensitive data in plain text.
Information leaks	Turn off PHP error display on the live site. Show friendly 403, 404, and 500 pages. Keep database passwords in config/database.php and do not upload that file to a public repository.
Unsafe connection	HTTPS only on the live site. Add basic security headers (X-Frame-Options, X-Content-Type-Options).

Also required

- **Input validation** on both frontend (for friendly messages) and backend (for real protection): required fields, length, format, and allowed values.
- **Audit logs** for login success and failure, logout, password changes and resets, account changes, enrollment decisions, grade encode/release/correction, clearance decisions, document status changes, payments, and settings changes.
- **File uploads are not included.** If added later, check the file type and size, and rename every file.
- **Data privacy:** collect only the data the system needs, and show each role only what it needs (this follows the Data Privacy Act of 2012, RA 10173).

14. UI/UX

Clean, modern, professional school-system look. One stylesheet and one set of components for every page.

- **Layout:** staff roles get a left sidebar with a top bar. Students get a simple top menu with large, clear buttons.
- **Responsive:** phone (up to 640 px), tablet (641–1024 px), desktop (above 1024 px). The sidebar becomes a menu button on small screens. Wide tables scroll sideways or become stacked cards.
- **Forms:** clear labels, helpful messages under the field, required marks, and no technical wording.
- **Tables:** search box, filters, sorting by main columns, and pagination (10–15 rows per page). Show a friendly message when there are no results.
- **Confirmation dialogs** before approve, reject, cancel, refund, and disable actions.
- **Messages:** green success and red error alerts after every action.
- **Loading state:** buttons show "Please wait…" and are disabled after clicking, so nothing is submitted twice.
- **Accessibility basics:** good color contrast, labeled inputs, and full keyboard use.

Status badge colors

Color	Statuses
Gray / amber	Pending, Draft, In Progress
Blue	Under Review, Pending Verification
Orange	For Payment, Unpaid, Action Needed
Purple/teal	Processing, Ready for Release
Green	Approved, Enrolled, Paid, Released, Completed, Cleared, Active
Red	Rejected, Cancelled, Refunded, Inactive

15. Reports

Role	Reports	Filters
Registrar	Enrollment count per program and status; document requests per type and status; grades released per term	Term, program, status, date range
Cashier	Payments per day; total collected; payments by status	Date range, status, method
Department	Pending clearance items: cleared vs. not cleared students	Program, year level
Admin	Users per role; activity summary; audit log	Role, user, action, date range

Reports can be printed from the browser, and the main tables can also be downloaded as CSV.

16. Performance and Reliability

- Pages load in **under 3 seconds** with about **50 users at the same time**.
- Large lists use pagination and database indexes.
- Works on the latest Chrome, Edge, Firefox, and on mobile Chrome and Safari.
- The live URL uses HTTPS and stays online until grading is finished.
- Export the database (schema.sql and seed.sql) before every deployment as a backup.

17. Important System Rules

- All modules use the **same database**. The system works as one integrated system, not separate modules.
- **No hardcoded** student records, grades, payments, requests, or statuses in the pages. Starter data lives only in seed.sql.
- Every important action updates the database, writes an audit log, and creates the right notification.
- Changes made by staff show right away on the related student and staff pages.
- Use consistent IDs and relationships. Validate on both frontend and backend.
- Users see only the information they are allowed to see.
- All add, edit, view, and status-change functions work. Search and filters work.
- Test accounts: at least one per role, plus several sample students, so every workflow can be tried from start to finish.